

AWS Notes

What is a Region in AWS?

👉 An **AWS Region** is a **geographical location** where AWS has multiple data centers.

Examples:

- us-east-1 (N. Virginia)
- ap-south-1 (Mumbai)
- eu-west-1 (Ireland)

1-line interview answer:

👉 A Region is a **physical geographic area** containing multiple Availability Zones.

What is an Availability Zone (AZ)?

👉 An **Availability Zone (AZ)** is an **isolated data center** within a Region.

- Each AZ has:
 - Independent power
 - Networking
 - Cooling
- AZs are connected via **high-speed, low-latency links**

Examples:

- ap-south-1a
- ap-south-1b
- ap-south-1c

1-line interview answer:

👉 An Availability Zone is a **physically separate data center** inside a Region.

IAM (Identity and Access Management)

IAM is used for authentication and authorization in AWS.

When we create an AWS account, we get a root user.

The root user has full access to all AWS services and resources.

In real-time scenarios, the DevOps/Admin creates the AWS account and never shares root credentials.

Instead, the admin creates IAM users for employees and grants only the required permissions (e.g., read data from a database but not delete it).

a) USERS

When a user needs access to AWS, the DevOps admin creates an IAM User.

IAM Users are used for authentication to get into AWS.

Each user gets credentials (username/password or access keys)

b) POLICIES : Whenever we create user, along with user we will attach some policies(What User Can perform inside AWS), If we are not attaching any policies then only user can enter into the aws, but he can't view and perform anything.

Simply, In Policies we Granting permissions for authorization.

c) GROUPS :

In Real time, there are particular set of people who are called developers and particular set of people who are called Testers

Groups like Developers, Testers, DBAdmins etc

Instead of attaching policies everytime for a new user, we will add the new user in a specific group based on his request.

Policies are attached to groups instead of individual users, This avoids attaching policies every time a new user is created.

d) ROLES :

Roles do not have permanent credentials

Roles are used to grant temporary access

Commonly used by:

AWS services (EC2, Lambda, etc.)

Cross-account access

Applications using AWS resources

Example:

An EC2 instance assumes a role to access S3 without storing credentials.

Why are IAM Roles preferred over access keys?

🔑 Roles improve security by eliminating hard-coded credentials.

Where are IAM Roles commonly used?

👉 With AWS services like **EC2, Lambda**, and for **cross-account access**.

Whenever IAM user is created, By default IAMUserChangePassword policy will be attached.

👉 **An IAM user can create other IAM users and attach permissions only if it has the required IAM policies.**

By default, an IAM user cannot create users.

The admin must attach policies like:

iam:CreateUser

iam:AttachUserPolicy

iam:PutUserPolicy

iam:AddUserToGroup

What is an IAM Policy?

👉 A policy is a **JSON document** that defines what actions are allowed or denied on AWS resources.

We can define our own custom policies also if we are not satisfied with AWS Managed policies.

Ex: *Custom policy like Only specific buckets in S3 not all buckets.*

👉 **IAM users are for:**

- AWS Console access
- CLI access (aws configure)
- Human users

👉 **IAM roles are for:**

- EC2, Lambda, ECS
- Cross-account access
- Temporary credentials

EC2 (ELASTIC COMPUTE CLOUD)

👉 Physical machines are converted into virtual servers using **virtualization**, and **AWS handles this in the background**.

❏ **What actually happens?**

A **physical server** (CPU, RAM, disk, network) is divided into multiple **virtual machines (EC2 instances)** using a **hypervisor**.

Physical Server

└— EC2 Instance 1

└— EC2 Instance 2

└— EC2 Instance 3

Each EC2 instance behaves like an **independent server**.

What is a Hypervisor? (Simple)

👉 A **hypervisor** is software that allows **multiple virtual machines (VMs)** to run on a **single physical machine** by sharing its hardware resources.

Who manages virtualization in AWS?

👉 AWS manages virtualization using its **hypervisor (Nitro/Xen)**.

Elastic means scale up or down based on demand.

👉 **EC2 (Elastic Compute Cloud)** is an AWS service where we request AWS to provide a **virtual machine** on their cloud platform, which can **scale up or down (elastic)** based on demand.

Types of EC2 instances (Nothing but Instance Types section in AWS):

📦 General Purpose (T, M)

Balanced CPU, memory, and network

Examples: t3, t3.micro, t3a, m5, m6

Interview line:

👉 Used when you need a **balance of compute, memory, and networking**. (Web applications)

📦 Compute Optimized (C)

High CPU performance

Examples: c5, c6

Interview line:

👉 Best for **compute-heavy workloads**. (Batch processing, gaming servers)

📦 Memory Optimized (R, X)

Large RAM

Examples: r5, r6, x1

Interview line:

👉 Best for applications that **require large memory**. (Caching, Real-time analytics)

📦 Storage Optimized (I, D)

High disk I/O

Examples: i3, i4, d2

Interview line:

👉 Used for workloads with **high disk read/write operations**. (Log processing, NOSQL Databases)

📺 **GPU / Accelerated Computing (P, G)**

High graphics & parallel processing

Examples: p3, p4, g4

Interview line:

👉 Used for **graphics-intensive and ML workloads**. (Machine Learning, AI model training)

Instance Type	Best For
General Purpose	Balanced workloads
Compute Optimized	High CPU
Memory Optimized	High RAM
Storage Optimized	High Disk I/O
GPU	ML / AI / Graphics

Important EC2 terminologies in AWS :

AMI (Amazon Machine Image):

👉 An AMI is a **template that contains the OS and software** used to create an EC2 instance.

Key Value Pair (Key Pair):

👉 A key pair is used to **securely connect to an EC2 instance** using SSH.

Instance Type:

👉 Instance type defines the **CPU, memory, storage, and network capacity** of an EC2 instance.

Security Group:

👉 A security group is a **virtual firewall** that controls inbound and outbound traffic for an EC2 instance.

SSH (Secure Shell):

👉 SSH is a protocol used to **securely connect to a remote server** using a command line.

Deploy Spring Boot App In EC2 :

📌 **Launch EC2 Instance**

- **AMI:** Amazon Linux 2 / Amazon Linux 2023
- **Instance type:** t2.micro or t3.micro
- **Key pair:** Create/download .pem

- **Security Group:**
 - 22 → SSH
 - 8080 → Spring Boot (or 80 if using Nginx)

2. Connect to EC2 via SSH

- `ssh -i key.pem ec2-user@<EC2-PUBLIC-IP>`

3. Install Java in EC2

4. Build Spring Boot JAR in Local Machine and copy JAR to EC2

Creates JAR File : `mvn clean package`

Copy JAR to EC2 : `scp -i key.pem target/app.jar ec2-user@<EC2-PUBLIC-IP>:/home/ec2-user/`

5. Run Spring Boot App in EC2

`java -jar app.jar`

👉 App runs on:

`http://<EC2-IP>:8080`

🔖 **Run App in Background (Important) :** `nohup java -jar app.jar > app.log 2>&1 &`

How do you deploy Spring Boot on EC2?

👉 Build a JAR, copy it to EC2, install Java, and run it using `java -jar`.

How do you keep the app running after logout?

👉 Use `nohup` or run it as a **systemd service**.

Which port does Spring Boot run on?

👉 Default port is **8080**.

Use **Putty** or **MobaXtrem** to connect to EC2 in Windows.

S3 (SIMPLE STORAGE SERVICE)

👉 Amazon S3 is an **object storage service** used to store and retrieve **any amount of data** from anywhere.

Objects can be image, video, audio, log files, csv etc

Key Concepts :

Bucket

👉 A bucket is a **container** used to store objects in S3.

- Bucket names are **globally unique**

Object

👉 A Object is a **file** stored in S3.

Objects can be image, video, audio, log files, csv etc

Key

👉 A key is the **unique name/path** of an object inside a bucket.

Scalability

👉 S3 is **infinitely scalable** — no size limits.

S3 store almost unlimited Data in a Single bucket, However one object should not be more than 5TB.

NOTE :

👉 Data is automatically replicated across **multiple AZs**.

🔒 Security (S3)

👉 S3 security controls **who can access the bucket and objects** and **how data is protected** using permissions and encryption.

👤 IAM Policies

👉 IAM policies define **what actions a user/role can perform on S3 resources**.

- Attached to **users, groups, or roles**
- Example: allow s3:GetObject only

📁 Bucket Policies

👉 Bucket policies define **who can access a specific S3 bucket** and what actions they can perform.

- Resource-based policy
- Commonly used for **public access or cross-account access**

S3 Access Control

IAM Policy

👉 Controls who can access S3 at the **user/role level**.

Bucket Policy

👉 Controls who can access the **bucket**.

🔒 Encryption in S3

👉 Encryption at rest ensures that **stored data in S3 is unreadable without the decryption key.**

👉 It means **data stored inside S3 is saved in an unreadable (encrypted) format** so that even if someone accesses the storage, they **cannot understand the data without the encryption key.**

What is “data at rest”?

👉 Data that is **stored and not moving.**

Examples: Files saved in S3, Backups, Logs

Term	Meaning
Data at rest	Stored data (S3 objects)
Data in transit	Data moving over network (HTTPS)

Does AWS encrypt S3 by default?

👉 Yes, S3 encrypts data at rest by default using **SSE-S3.**

💎 SSE-S3

👉 Server-Side Encryption managed **entirely by AWS** using S3-managed keys.

- Simple
- No key management

💎 SSE-KMS

👉 Server-Side Encryption using **AWS KMS customer-managed keys.**

- More control
- Audit via CloudTrail

Encryption

At Rest:

- SSE-S3 (AWS managed)
- SSE-KMS (Customer managed keys)

In Transit:

- HTTPS

S3 Storage Classes

Storage Class	Use Case
Standard	Frequently accessed data
Intelligent-Tiering	Unknown access pattern
Standard-IA	Infrequent access
One Zone-IA	Non-critical, single AZ
Glacier Instant	Archive with fast access
Glacier Flexible	Archive (minutes–hours)
Glacier Deep Archive	Long-term backup

We can enable **Versioning** also in S3

Versioning

👉 Versioning keeps **multiple versions** of an object to protect against accidental deletion.

S3 Event Notifications

👉 S3 can trigger **Lambda, SNS, or SQS** on object events.

Static Website Hosting

👉 S3 can host **static websites** (HTML, CSS, JS).

Q: Is S3 global or regional?

👉 S3 is a **global service**, but buckets are **region-specific**.

We created role and attached role to EC2 instance, so that we are able access S3 from ec2

Created role and attached s3 full access policy.

High-level idea (1 line)

👉 A Spring Boot app connects to S3 using the **AWS SDK**, authenticated via **IAM credentials or IAM Role**.

Ways to Authenticate (VERY IMPORTANT)

✓ Recommended (Production)

👉 IAM Role attached to EC2 / ECS / EKS

- No access keys in code
- Most secure
- AWS automatically provides credentials

⚠ Not Recommended (Local / Testing)

👉 Access Key + Secret Key

1) Connecting to S3 from local Spring Boot App

1️⃣ Create IAM User (One-time)

- Go to **IAM → Users**
- Attach policy:
 - AmazonS3FullAccess (*for learning*)
 - or custom policy with s3:GetObject, s3:PutObject
- Generate:
 - **Access Key**
 - **Secret Key**

2️⃣ Configure Credentials Locally

Option A ✓ (Recommended): AWS CLI

aws configure

Enter:

AWS Access Key ID

AWS Secret Access Key

Default region

Stored in:

~/.aws/credentials

~/.aws/config

Option B : Storing in spring boot app

application.properties

```
aws.access-key=AKIAxxxxxxxxxxxxx
aws.secret-key=xxxxxxxxxxxxxxxxx
aws.region=ap-south-1
aws.s3.bucket-name=my-s3-bucket
```

3 Add AWS SDK Dependency

```
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>s3</artifactId>
</dependency>
```

4 Configure S3 Client in Spring Boot

👉 This reads credentials from ~/.aws/credentials

@Configuration

```
public class S3Config {

    @Bean
    public S3Client s3Client() {
        return S3Client.builder()
            .region(Region.AP_SOUTH_1)
            .credentialsProvider(ProfileCredentialsProvider.create())
            .build();
    }
}
```

👉 This reads credentials from Properties File

@Configuration

```
public class S3Config {

    @Value("${aws.access-key}")
    private String accessKey;

    @Value("${aws.secret-key}")
    private String secretKey;

    @Value("${aws.region}")
    private String region;
```

@Bean

```
public S3Client s3Client() {  
  
    AwsBasicCredentials awsCredentials =  
        AwsBasicCredentials.create(accessKey, secretKey);  
  
    return S3Client.builder()  
        .region(Region.of(region))  
        .credentialsProvider(StaticCredentialsProvider.create(awsCredentials))  
        .build();  
}  
}
```

S3Controller Class

@RestController

@RequestMapping("/s3")

```
public class S3Controller {  
    private final S3Service s3Service;  
  
    public S3Controller(S3Service s3Service) {  
        this.s3Service = s3Service;  
    }  
  
    @PostMapping("/upload")  
    public ResponseEntity<String> upload(@RequestParam MultipartFile file) throws IOException {  
        s3Service.uploadFile(file);  
        return ResponseEntity.ok("File uploaded successfully");  
    }  
  
    @GetMapping("/download/{key}")  
    public ResponseEntity<byte[]> download(@PathVariable String key) {  
        byte[] data = s3Service.downloadFile(key);  
        return ResponseEntity.ok()  
            .header(HttpHeaders.CONTENT_DISPOSITION,  
                "attachment; filename=\"" + key + "\"")  
            .body(data);  
    }  
}
```

```
}  
}
```

S3Service Class

@Service

```
public class S3Service {  
    private final S3Client s3Client;  
    @Value("${aws.s3.bucket-name}")  
    private String bucketName;  
    public S3Service(S3Client s3Client) {  
        this.s3Client = s3Client;  
    }  
  
    // Upload file  
    public void uploadFile(MultipartFile file) throws IOException {  
        s3Client.putObject(  
            PutObjectRequest.builder()  
                .bucket(bucketName)  
                .key(file.getOriginalFilename())  
                .build(),  
            RequestBody.fromBytes(file.getBytes())  
        );  
    }  
  
    // Download file  
    public byte[] downloadFile(String key) {  
  
        ResponseBytes<GetObjectResponse> objectAsBytes =  
            s3Client.getObjectAsBytes(  
                GetObjectRequest.builder()  
                    .bucket(bucketName)  
                    .key(key)  
                    .build()  
            )  
    }  
}
```

```

    );
    return objectAsBytes.asByteArray();
}
}

```

2) Connecting to S3 from AWS Services(EC2/ECS/EKS)

✓ Recommended (Production)

👉 IAM Role attached to EC2 / ECS / EKS

📄 Create an IAM Role

- Trusted entity: **AWS Service**
- Service : EC2
- Attach policy: AmazonS3FullAccess (or least-privilege S3 policy)

📄 Attach the IAM Role to the EC2 instance

📄 Spring Boot app running on EC2 automatically uses this role

- No access key
- No secret key
- AWS SDK fetches temporary credentials

📄 Add AWS SDK Dependency

```

<dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>s3</artifactId>
</dependency>

```

📄 Configure S3 Client (Spring Bean)

@Configuration

```

public class S3Config {

    @Bean
    public S3Client s3Client() {
        return S3Client.builder()
            .region(Region.AP_SOUTH_1)
            .build(); // Credentials auto-picked from IAM Role
    }
}

```

```
}  
}
```

👉 AWS SDK automatically checks:

1. IAM Role
2. Environment variables
3. AWS credentials file

📁 Upload File to S3

@Service

```
public class S3Service {  
    private final S3Client s3Client;  
    public S3Service(S3Client s3Client) {  
        this.s3Client = s3Client;  
    }  
    public void uploadFile(String bucketName, String key, Path filePath) {  
        PutObjectRequest request = PutObjectRequest.builder()  
            .bucket(bucketName)  
            .key(key)  
            .build();  
        s3Client.putObject(request, filePath);  
    }  
}
```

📁 Download File from S3

```
public void downloadFile(String bucketName, String key, Path destination) {  
    GetObjectRequest request = GetObjectRequest.builder()  
        .bucket(bucketName)  
        .key(key)  
        .build();  
    s3Client.getObject(request, destination);  
}
```

IAM Permission Required (Interview MUST)

```
{
```

```
"Effect": "Allow",  
"Action": [  
  "s3:GetObject",  
  "s3:PutObject"  
],  
"Resource": "arn:aws:s3:::my-bucket/*"  
}
```

How it works in EC2 (Real-Time Flow)

1. Spring Boot runs on EC2
2. EC2 has an **IAM Role**
3. Role has **S3 permissions**
4. AWS SDK fetches **temporary credentials**
5. App accesses S3 securely

👉 **No keys in code**

VPC, NSG & NACL

VPC (Virtual Private Cloud)

👉 A **VPC** is your **own private network** inside AWS where you launch resources like EC2 and RDS.

Simple analogy:

🏠 VPC = Your house

Inside VPC, We having Subnets :

Public Subnet

👉 A **public subnet** is a subnet where resources **can access the internet directly**.

Key point:

- Has a **route to an Internet Gateway (IGW)**

Private Subnet

👉 A **private subnet** is a subnet where resources **cannot access the internet directly**.

Key point:

- **No direct route** to Internet Gateway
- Uses **NAT Gateway** if internet access is needed

- **Public Subnet:** Subnet with internet access via IGW(Internet Gateway)
- **Private Subnet:** Subnet without direct internet access

Inbound

👉 **Inbound traffic means incoming traffic** — data coming **into** your server.

Example:

- User opens a website
- Request comes **from internet → EC2**

Outbound

👉 **Outbound traffic means outgoing traffic** — data going **out from** your server.

Example:

- EC2 sends response back
- EC2 connects to database or internet

Security Group

👉 A **Security Group** is a **virtual firewall** that controls **who can access a resource and on which port**.

Simple analogy:

📖 Security Group = Door security / bouncer

- Works at **instance level**
- Allows traffic only
- **Stateful**

💎 Security Group (SG)

- You define **inbound and outbound rules**
- **Stateful**
 - If inbound traffic is allowed, outbound response is **automatically allowed**
- You **only allow** traffic (no deny rules)

NACL (Network Access Control List)

👉 A **NACL** is a firewall that controls traffic **at the subnet level**.

Simple analogy:

🏠 NACL = Society gate security

- Works at **subnet level**
- Allows and denies traffic
- **Stateless**

🔒 NACL

- You define **inbound and outbound rules**
- **Stateless**
 - You must explicitly allow **both request and response**
- You can **allow and deny** traffic

Concept	Simple Meaning	Level
VPC	Private network	Network
Security Group	Instance firewall	Instance
NACL	Subnet firewall	Subnet

AMAZON RDS

What is RDS ?

👉 Amazon RDS is a **managed relational database service** where AWS handles backups, patching, and availability.

Databases Supported

- MySQL
- PostgreSQL
- Oracle
- SQL Server
- MariaDB
- **Amazon Aurora** (AWS-optimized) : MySQL and PostgreSQL Compatible only

Key RDS Concepts (Must-Know)

DB Instance

👉 A DB instance is a **database server** running in AWS.

Storage

- SSD (gp3, io1)
- Auto-scaling storage supported

Backups

- Automated backups (point-in-time recovery)
 - Manual snapshots
-

Multi-AZ

👉 Multi-AZ provides **high availability** by creating a standby DB in another AZ.

- Automatic failover
 - No manual intervention
-

Read Replicas

👉 Read replicas are used for **read scaling**, not failover.

Security

- IAM authentication (optional)
 - VPC isolation
 - Security Groups
 - Encryption at rest & in transit
-

Availability & Durability

- Multi-AZ → High availability
- Backups → Data protection

RDS Best Practices

- Use **Multi-AZ** for prod
- Enable **automated backups**
- Do NOT make DB public
- Use **least-privilege security groups**

Connecting RDS from Spring Boot (Real-Time)

📦 Create RDS Instance

- Engine: MySQL / PostgreSQL

- VPC: Same as EC2
- Public access: ✗ No (recommended)
- Security Group:
 - Allow DB port from **EC2 security group**

2 Security Group Setup (Very Important)

Source	Port
EC2 SG	3306 (MySQL)
EC2 SG	5432 (Postgres)

3 Spring Boot Dependencies

MySQL

<dependency>

<groupId>com.mysql</groupId>

<artifactId>mysql-connector-j</artifactId>

</dependency>

4 application.properties

spring.datasource.url=**jdbc:mysql://rds-endpoint:3306/mydb**

spring.datasource.username=dbuser

spring.datasource.password=dbpassword

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

👉 Use **RDS endpoint**, not IP.

5 How Connection Works (Real-Time Flow)

Spring Boot App



EC2 Instance



Security Group



RDS Endpoint

👉 In real time, the Spring Boot app running on EC2 connects to the RDS endpoint only if the EC2 security group allows outbound traffic and the RDS security group allows inbound access from the EC2 security group.

📁 Production Best Practices

✅ Use Secrets Manager

- Store DB credentials securely

✅ Use Connection Pool

- HikariCP (default in Spring Boot)

✅ Enable SSL

useSSL=true

Difference between Multi-AZ and Read Replica?

👉 Multi-AZ is for high availability, Read Replica is for read scaling.

Can we SSH into RDS?

👉 No, RDS is fully managed.

How does Spring Boot connect to RDS?

👉 Using JDBC with the RDS endpoint and security group access.

Connecting RDS from Local Spring Boot App

📁 RDS Side Configuration (MOST IMPORTANT)

✅ Make RDS Publicly Accessible

- While creating RDS:
 - **Public access:** Yes (*for local access only*)

✅ Configure RDS Security Group

Allow DB access **from your local machine IP**:

Type	Port	Source
MySQL	3306	Your public IP (x.x.x.x/32)
PostgreSQL	5432	Your public IP (x.x.x.x/32)

🔗 Get RDS Endpoint

Example: mydb.c9akciq.us-east-1.rds.amazonaws.com

📁 application.properties (Local App)

spring.datasource.url=jdbc:mysql://mydb.c9akciq.us-east-1.rds.amazonaws.com:3306/mydb

spring.datasource.username=dbuser

spring.datasource.password=dbpassword

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

How Connection Works (1-line flow)

👉 Local Spring Boot app → Internet → RDS endpoint → Security Group allows access.

How does a local Spring Boot app connect to RDS?

👉 Using the public RDS endpoint with proper security group rules.

Best Practice

👉 For production, run Spring Boot on **EC2/ECS inside the VPC** and keep RDS **private**.

LAMBDA FUNCTION

What is AWS Lambda (Simple words)

👉 AWS Lambda is a serverless compute service where you run code without managing servers and pay only for execution time.

Why it's called *Serverless*

- You **don't create EC2 instances**
- AWS handles:
 - Servers
 - OS
 - Scaling
 - Patching
- You focus **only on code**

How Lambda Works (Real-Time Flow)

Event (S3 / API / DynamoDB / CloudWatch)



AWS Lambda



Your Code Runs



Output / Response

Key Components (Interview Must-Know)

1 Function

- A piece of code (Java, Python, Node.js, etc.)
- Executes when triggered

2 Event Source (Trigger)

- What starts the Lambda
- Examples:
 - API Gateway (HTTP request)
 - S3 (file upload)
 - DynamoDB (data change)
 - CloudWatch (scheduled job)

3 Execution Role (IAM Role)

- Defines **what Lambda is allowed to access**
- Example:
 - Read from S3
 - Write to DynamoDB

Lambda Execution Model (Important 🧠)

Cold Start

- First invocation → AWS creates container
- Slight delay (Java has higher cold start)

Warm Start

- Container reused
- Faster execution

👉 **Interview tip:** Java Lambdas have higher cold start than Python/Node.

Lambda Limits (Very Important)

Item	Limit
Max execution time	15 minutes
Memory	128 MB – 10 GB

Item	Limit
Storage	/tmp → 512 MB
Concurrent executions	Account limit

Pricing

👉 You pay for:

- **Number of executions**
- **Execution duration (ms)**

✗ No cost when Lambda is idle

When to Use Lambda (Use Cases)

- REST APIs (via API Gateway)
- File processing (S3 uploads)
- Event-driven systems
- Scheduled jobs (cron)
- Lightweight microservices

When NOT to Use Lambda ✗

- Long-running jobs (>15 min)
- Heavy CPU workloads
- Stateful applications
- Low-latency Java services

Lambda vs EC2 (1-Line Difference)

👉 Lambda is event-driven and serverless, while EC2 is server-based and requires infrastructure management.

Q: Can Lambda access VPC resources?

👉 Yes, by attaching Lambda to a VPC.

Q: Can Lambda connect to RDS?

👉 Yes, but connection pooling is required (RDS Proxy).

Q: Is Lambda stateless?

👉 Yes, each invocation should be stateless.

Final 1-Line Summary (Perfect Ending)

👉 AWS Lambda is a serverless, event-driven compute service that automatically scales and charges only for execution time.

End-to-End code flow to call an AWS Lambda from a local Spring Boot application

🔄 Overall Flow (Understand First)

Local Spring Boot App



AWS SDK (Lambda Client)



IAM Authentication



AWS Lambda Function



Response back

📋 Prerequisites (VERY IMPORTANT)

✅ Lambda Function Exists

- Example Lambda name: hello-lambda
- Runtime: Java / Python / Node (doesn't matter)

✅ IAM User Permissions

IAM user must have:

```
{  
  "Effect": "Allow",  
  "Action": "lambda:InvokeFunction",  
  "Resource": "*" }  
}
```

📦 Maven Dependencies (Spring Boot)

<dependency>

```
<groupId>software.amazon.awssdk</groupId>
<artifactId>lambda</artifactId>
</dependency>
```

```
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>auth</artifactId>
</dependency>
```

📦Application.properties (Local)

```
aws.access-key=YOUR_ACCESS_KEY
aws.secret-key=YOUR_SECRET_KEY
aws.region=us-east-1
aws.lambda.function-name=hello-lambda
```

🔧AWS Lambda Client Configuration

@Configuration

```
public class AwsLambdaConfig {
    @Value("${aws.access-key}")
    private String accessKey;
    @Value("${aws.secret-key}")
    private String secretKey;
    @Value("${aws.region}")
    private String region;
```

@Bean

```
public LambdaClient lambdaClient() {
    return LambdaClient.builder()
        .region(Region.of(region))
        .credentialsProvider(
            StaticCredentialsProvider.create(
                AwsBasicCredentials.create(accessKey, secretKey)
            )
        )
}
```

```

    )
    .build();
}

```

5 Service Class to Invoke Lambda

```

@Service

public class LambdaService {

    @Value("${aws.lambda.function-name}")
    private String functionName;

    private final LambdaClient lambdaClient;

    public LambdaService(LambdaClient lambdaClient) {
        this.lambdaClient = lambdaClient;
    }

    public String invokeLambda(String input) {

        InvokeRequest request = InvokeRequest.builder()

            .functionName(functionName)

            .payload(SdkBytes.fromUtf8String(input))

            .build();

        InvokeResponse response = lambdaClient.invoke(request);

        return response.payload().asUtf8String();
    }
}

```

6 REST Controller (Trigger from Browser / Postman)

```

@RestController

@RequestMapping("/lambda")

public class LambdaController {

    private final LambdaService lambdaService;

    public LambdaController(LambdaService lambdaService) {
        this.lambdaService = lambdaService;
    }

    @GetMapping("/call")

```

```

public String callLambda() {
    return lambdaService.invokeLambda("{\"message\":\"Hello from Spring Boot\"}");
}
}

```

Final Execution Flow

👉 Browser/Postman → Spring Boot Controller → AWS SDK → IAM Auth → Lambda → Response

Q: Does Lambda need Security Group?

👉 No, Lambda invocation is controlled by **IAM**, not SG.

Q: Can Lambda be private?

👉 Yes, Lambda does not need to be public.

Q: Is API Gateway required?

👉 No, SDK can invoke Lambda directly.

Production Best Practices

- Use **IAM Role** instead of access keys
- Use **AWS profiles or env variables**
- Enable **timeouts & retries**
- Use **Async invocation** if needed

CLOUD WATCH

In simple words, Cloud watch means Watching the activities in the cloud.

👉 CloudWatch monitors AWS resources and applications by collecting metrics, logs, and triggering alerts.

Core Components (Very Important for Interviews)

Metrics

- Numerical data points
- Examples:
 - EC2 CPU usage
 - RDS connections
 - Lambda errors
 - S3 request count

👉 Example:

If EC2 CPU > 80% → trigger alarm

Logs

- Application logs
- System logs
- Lambda logs
- Custom logs from Spring Boot



Real-time example:

Your Spring Boot app running on EC2 pushes logs to CloudWatch Logs for debugging.

Alarms

- Trigger notifications based on thresholds
- Integrated with:
 - SNS
 - Auto Scaling
 - Lambda



Example:

CPU > 80% → Send email via SNS

Events (EventBridge Integration)

- Trigger actions based on AWS events
- Example:
 - EC2 state change
 - Scheduled cron job



Real-Time Use Cases

Use Case 1: Auto Scaling

EC2 CPU > 75%

→ CloudWatch Alarm

→ Auto Scaling adds new EC2

Use Case 2: Production Alerting

RDS free storage < 10%

→ CloudWatch Alarm

→ SNS sends email to DevOps team

Common Cost Optimization Use Cases(Using CloudWatch & Lambda)

✓ Stop Unused EC2 Instances

Problem: EC2 running but not used

Detection:

- CloudWatch monitors CPU Utilization < 5% for 1 hour

Action:

- Lambda stops the EC2 instance

Interview line:

👉 Use CloudWatch CPU metrics and Lambda to stop idle EC2 instances.

✓ Auto-Delete Old Logs

Problem: CloudWatch logs growing endlessly

Detection:

- CloudWatch log groups older than X days

Action:

- Lambda deletes or reduces retention

✓ S3 Cost Control

Problem: Old data in expensive S3 class

Detection:

- CloudWatch monitors S3 storage metrics

Action:

- Lambda applies lifecycle rules (move to Glacier)

👉 Glacier => Moving data from regular S3 storage to a cheaper long-term storage class meant for rarely accessed data.

In Simple Words

- Frequently used data → Keep in **S3 Standard**
- Rarely used data → Move to **Glacier** (very cheap storage)

✗ What CloudWatch Alone Cannot Do

- It **detects**, but does not **act**

👉 Lambda is needed to **take action**

👉 CloudWatch monitors resource usage and triggers Lambda functions to automatically stop, resize, or clean up unused AWS resources for cost optimization.

□ The Core Idea

You want to solve:

“CloudWatch logs are growing endlessly — clean them automatically”

This is:

- **Not real-time**
- **Not metric-based**
- **Needs periodic checking**

👉 That’s exactly where **scheduler + automation** fits.

⚙️ Components Involved

1. Amazon EventBridge

- Works like a **cron job**
 - Triggers tasks at fixed intervals (e.g., daily)
-

2. AWS Lambda

- Runs your cleanup logic
 - No server needed
 - Executes only when triggered
-

3. Amazon CloudWatch

- Stores all your logs
- Where cleanup is required

🔄 End-to-End Flow

EventBridge (runs every day at 2 AM)



Triggers Lambda



Lambda fetches all CloudWatch log groups



Checks:

- retention policy
- age of logs



If needed:

- Set retention (e.g., 30 days)

OR

- Delete unused log groups

What Lambda Actually Does (Step-by-Step)

Inside Lambda:

1. Call AWS API → DescribeLogGroups
2. Loop through each log group
3. Check:
 - If retention is missing OR too high
4. Apply:
 - PutRetentionPolicy(30 days)

 Done automatically for all services

Why This is “Industry Standard”

1. Separation of Concerns

- CloudWatch → just stores logs
- Lambda → handles logic
- EventBridge → handles scheduling

 Clean architecture

DYNAMO DB

What is DynamoDB (Simple Words)

 DynamoDB is a fully managed NoSQL database service in AWS designed for high performance and automatic scaling.

1-Line Interview Answer

 DynamoDB is a serverless, key-value and document NoSQL database that provides single-digit millisecond performance at any scale.

Core Concepts (VERY Important for Interviews)

 Table

- Similar to RDBMS table
 - Stores items
-

Item

- Similar to a row
 - Collection of attributes
-

Attribute

- Similar to a column
 - Key-value pair
-

Primary Key (VERY IMPORTANT 🙌)

Partition Key

- Unique identifier
- Determines data distribution

Composite Key

- Partition Key + Sort Key
- Allows multiple items per partition

👉 **Interview Tip:** Proper partition key design is critical for performance.

How Data is Distributed

- DynamoDB uses **partitioning**
- Based on **partition key hash**
- Automatically scales horizontally

Scaling Options

On-Demand Mode

- Auto scales
- Pay per request
- Good for unpredictable workloads

Provisioned Mode

- Define read/write capacity
 - Cheaper for predictable workloads
-

Performance

- Single-digit millisecond latency
- Fully managed
- No server management

Important Features (Interview Must-Know)

Secondary Indexes

Global Secondary Index (GSI)

- Different partition key
- Flexible querying

Local Secondary Index (LSI)

- Same partition key
- Different sort key

Secondary Indexes in Amazon DynamoDB

 Secondary indexes allow you to query DynamoDB data using attributes other than the primary key.

Global Secondary Index (GSI)

 A **GSI** lets you query data using a **different partition key (and optional sort key)** than the base table.

- Supports **flexible querying**
- Can be added **after table creation**
- Has its **own read/write capacity**

Interview line:

 GSI is used when you need to query DynamoDB using different access patterns.

Example Scenario

Base Table

Partition Key: userId

Sort Key: orderId

You can query: Get all orders for userId = 101

✓ This works because userId is the partition key.

New Requirement

Now business asks:

👉 "Get all orders by orderStatus = SHIPPED"

But orderStatus is **not part of the primary key** ✗
So DynamoDB cannot query efficiently.

✓ Solution: GSI

Create a **GSI**:

Partition Key: orderStatus

Sort Key: orderDate

Now you can query: Get all SHIPPED orders

So What is "Different Access Pattern"?

It means:

Access Pattern	Example
Query by userId	Base table
Query by orderStatus	GSI
Query by email	GSI
Query by createdAt	GSI

👉 Each new way of querying = new access pattern.

Interview-Level Explanation

👉 In DynamoDB, we design tables based on how data is accessed. If the application needs to query using attributes other than the primary key, we create a GSI to support those access patterns.

◆ Local Secondary Index (LSI)

👉 An **LSI** lets you query data using the **same partition key but a different sort key**.

- Must be created **at table creation**
- Shares capacity with the base table

Interview line:

👉 LSI is used when you need multiple sort orders within the same partition.

◆ Local Secondary Index (LSI) – Example

 **Base Table: Orders**

Primary Key

- Partition Key: userId
- Sort Key: orderId

👉 This allows:

Get all orders for userId = 101 ordered by orderId

🔔 New Requirement

👉 "For the same user, list orders sorted by orderDate."

- Partition key stays the same → userId
- Sort order needs to change → ❌ Not possible with base table

✅ Solution: LSI

Create an **LSI** at table creation:

LSI Key

- Partition Key: userId (same)
- Sort Key: orderDate (different)

✅ Now You Can Query

Get all orders for userId = 101 ordered by orderDate

Interview Summary (1 line)

👉 LSI is used when you need different sorting options for items that belong to the same partition key.

Quick Comparison Reminder ☐

Requirement	Use
Same partition, different sort	LSI
Different partition key	GSI

💎 Streams

- Captures changes in table
 - Used with Lambda for event-driven systems
-

💎 TTL (Time To Live)

- Auto-delete expired items
 - Useful for session data
-

💎 Transactions

- Supports ACID transactions

🔗 DynamoDB vs RDS (1-Line Difference)

👉 DynamoDB is NoSQL and highly scalable, while RDS is relational and supports SQL with joins.

👤 Common Interview Questions

Q: What happens if partition key is poorly designed?

👉 Hot partition → throttling → performance issues

Q: Does DynamoDB support joins?

👉 No

Q: Can it scale automatically?

👉 Yes, especially in On-Demand mode

🚀 Connect to Amazon DynamoDB from Local Spring Boot

🔄 High-Level Flow

Local Spring Boot App



AWS SDK (DynamoDB Client)



IAM Access Key / Secret Key



DynamoDB Table

✅ Step 1: Create IAM User

1. Go to IAM in AWS
2. Create a user
3. Enable **Programmatic access**

4. Attach policy: AmazonDynamoDBFullAccess
5. Save :
 - Access Key
 - Secret Key

✔ Step 2: Add Maven Dependencies

```
<dependency>

    <groupId>software.amazon.awssdk</groupId>

    <artifactId>dynamodb</artifactId>

</dependency>

<dependency>

    <groupId>software.amazon.awssdk</groupId>

    <artifactId>auth</artifactId>

</dependency>
```

✔ Step 3: Configure application.properties

```
aws.access-key=YOUR_ACCESS_KEY

aws.secret-key=YOUR_SECRET_KEY

aws.region=us-east-1
```

✔ Step 4: Create DynamoDB Client Bean

```
@Configuration

public class DynamoDBConfig {

    @Value("${aws.access-key}")

    private String accessKey;

    @Value("${aws.secret-key}")

    private String secretKey;

    @Value("${aws.region}")

    private String region;

    @Bean

    public DynamoDbClient dynamoDbClient() {

        return DynamoDbClient.builder()

            .region(Region.of(region))

            .credentialsProvider(
```

```

        StaticCredentialsProvider.create(
            AwsBasicCredentials.create(accessKey, secretKey)
        )
    )
    .build();
}
}

```

✓ Step 5: Create Service to Insert Data

@Service

```

public class DynamoService {

    private final DynamoDbClient dynamoDbClient;

    public DynamoService(DynamoDbClient dynamoDbClient) {
        this.dynamoDbClient = dynamoDbClient;
    }

    public void saveItem() {
        Map<String, AttributeValue> item = new HashMap<>();
        item.put("id", AttributeValue.builder().s("1").build());
        item.put("name", AttributeValue.builder().s("Sai").build());

        PutItemRequest request = PutItemRequest.builder()
            .tableName("users")
            .item(item)
            .build();

        dynamoDbClient.putItem(request);
    }
}

```

Q: Does DynamoDB require Security Groups?

👉 No (It's a managed AWS service accessed via IAM)

Q: Can local app connect directly?

👉 Yes, using IAM credentials.

Q: Is VPC required?

👉 Not for public endpoint access.

Connect to Amazon DynamoDB from Spring Boot running in Amazon EC2

1-Line Interview Answer

👉 When Spring Boot runs on EC2, it connects to DynamoDB using an IAM Role attached to the EC2 instance — no access keys required.

High-Level Flow

Spring Boot App (EC2)



IAM Role (Instance Profile)



AWS SDK (Default Credentials)



DynamoDB Table

Step 1: Create IAM Role for EC2

1. Go to IAM → Roles
2. Select **EC2** as trusted entity
3. Attach policy:

For learning: AmazonDynamoDBFullAccess

For production (recommended minimal permissions):

```
{  
  "Effect": "Allow",  
  "Action": [  
    "dynamodb:PutItem",  
    "dynamodb:GetItem",  
    "dynamodb:Query",  
    "dynamodb:UpdateItem"  
  ],  
}
```

```
"Resource": "*"
}
```

✓ Step 2: Attach IAM Role to EC2

- Go to EC2 → Instances
- Select instance
- Actions → Security → Modify IAM Role
- Attach created role

✓ Step 3: Add Maven Dependency

```
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>dynamodb</artifactId>
</dependency>
```

✓ Step 4: Configure DynamoDB Client (NO KEYS NEEDED 🤖)

@Configuration

```
public class DynamoConfig {
    @Bean
    public DynamoDbClient dynamoDbClient() {
        return DynamoDbClient.builder()
            .region(Region.US_EAST_1)
            .build(); // Uses EC2 IAM Role automatically
    }
}
```

📖 AWS SDK automatically uses:

- Instance metadata service
- IAM Role credentials

No access key required.

✓ Step 5: We can use `DynamoDbClient` as required.

🔗 Do We Need Security Groups?

👉 No. DynamoDB is accessed via AWS internal APIs and IAM authentication.

You can improve security by:

- Using **VPC Endpoint for DynamoDB**
- Restricting IAM role permissions
- Enabling CloudWatch monitoring